

VIVA : VLM instructed VLA Architecture

Isaac sim 및 기본 환경 세팅은 Lekiwi 환경 구축 및 원격조종 물체 스폰 매뉴얼을 따라
진행되어있다고 판단하고 스킬별 expert 및 학습 데이터 생성은 Skill wise expert bases
data generation 매뉴얼을 통해 진행되어있다고 판단한다. 또한 전체 코드는
<https://app.notion.com/p/383072c6fe2281058f7eef39f429eac7> 페이지를 참고한다.

0. 실행환경

- Ubuntu 22.04.5 LTS
 - Conda (Miniconda)
 - 클라이언트 env_isaacclab : Python 3.11 / Isaac Sim 5.0.0 / Isaac Lab 0.44.9 / PyTorch 2.7.0+cu128
 - 서버 vllm : Python 3.11 / vLLM 0.17.0 / Qwen3-VL-8B-Instruct (VLM, 포트 8000)
 - 서버 lerobotpi0v2 : Python 3.12 / LeRobot 0.5.0 / Pi0.5 (VLA, 포트 8002)
 - GPU : 클라이언트 RTX + 드라이버 / 서버 A100 40GB (VLM ~29.8GB + VLA ~8GB)
-

1. 운영 아키텍처

[클라이언트 PC - 데스크탑/노트북]

Isaac Sim으로 로봇, 장면(ProctorH0R) 시뮬레이션 실행

매 스텝 카메라 이미지(base_cam, wrist_cam)를 서버로 보내고, 받은 action으로 로봇 동작

[A100 40GB 서버]

클라이언트와는 SSH 터널로 연결 (8000 = VLM, 8002 = VLA)

VLM, VLA 모델 추론용으로 활용

VLM 서버 (Qwen3-VL-8B, 포트 8000) : 카메라 이미지 + 유저 명령 -> 어떤 스킬을 할지 + 자연어 지시

VLA 서버 (Pi0.5, 포트 8002) : 카메라 이미지 + 자연어 지시 -> 로봇 action 50 스텝 chunk

2. 파이프라인 개요

VLA 는 카메라 이미지와 자연어 지시를 받아 별도의 파이프라인 없이 바로 연속 action 을 출력하는 정책으로 VLM 백본에서 물려받은 언어능력과 비전 이해 능력 덕에 단순한 grasp, lift 처럼 짧은 구간의 행동은 안정적으로 수행한다. 다만

navigate→approach&lift→carry→approach&place 로 이어지는 길고 복잡한 task 에서 지시문 하나로 end-to-end 학습한 단일 VLA 는 명확한 한계가 드러난다. 그 이유는 제한된 관측 환경에서 매 스텝 짧은 action chunk 를 내는 제어에 최적화되어 있어 명령이 길고 복잡해지면 이를 단계별로 분해하고 현재 수행도를 판단해

다음 단계로 넘어가는 상위 추론을 충분히 끌어내지 못하기 때문이다. 그래서 시퀀스가 길어질수록 작은 오차가 누적되고 물체가 가려지거나 동작을 실패 했을 때와 같은 변수 상황에서 회복하지 못하며 스킬 경계에서는 지금이 어느 단계인지 혼동해 엉뚱한 물체로 향하거나 전환 시점을 놓친다.

결국 단일 VLA 는 잘 움직이긴 해도 길게 계획하고 단계를 전환하는 데는 약하다.

VIVA 는 이 문제를 계층형 구조로 모델별로 역할을 나누는 방식으로 푼다. VLA 의 강한 저수준 반응형 제어는 그대로 두고 단일 VLA 에서 병목이던 상위 추론(명령 분해,스킬 전환, 재계획)만 떼어내 강한 언어 능력을 갖춘 별도의 상위 VLM 에 맡긴다. 상위 VLM 이 실시간으로 장면을 보고 현재 상황이 어떤지, 어떤 단계인지 등을 판단해 그 시점에 맞는 서브태스크 지시를 내려주면 하위 VLA 는 그 시점에 맞는 지시문만 입력 이미지를 기반으로 행동하면 된다. 이렇게 하나의 long-horizon task 를 여러 short-horizon 제어 문제로 쪼개면 각 구간은 VLA 가 안정적으로 해내고 오차 누적과 문맥 소실도 구간 단위로 끊겨 억제된다. 또한 상/하위가 독립된 모듈이라 각각의 모델을 때에 따라 교체하거나 다른 방식으로 교체, 재사용, 조합하기도 쉽다.

[흐름]

유저 명령 → classify(VLM 1회, 집을 물체 / 놓을 곳 추출) → 4-skill state 판단

각 스킬 : VLM orchestrator(비동기, vlm_interval 스텝마다 자연어 지시)

→ VLA(동기, action chunk) → env.step

전환 : S1→S2 / S3→S4 = VLM 이 TARGET_FOUND (중앙 depth > 1.2m 면 거부)

S2→S3 / S4→DONE = 코드 pose 판정 (들어올림 완료 / 그리퍼 열림+비접촉)

[세부사항]

VLM : vlm_interval 기본 30 (검증하며 조절 필요)

VLA : action chunk 50 개 버퍼링, n_use 만큼 실행 후 재쿼리 (navigate, carry -> 10 / approach&lift, approach&place -> 50)

action 9D : Pi0.5 32D 출력 → 앞 9D 사용 [arm0-4, gripper, base vx · vy(전진) · wz]

depth : 카메라 중앙 1/3 최소깊이 < safety_dist(0.3m)면 전진 차단 (S1, S3 만)

카메라 해상도 : VLM 1280x800 / VLA 640x400

[실행 모드별 체크포인트]

모드	VLM	VLA	로컬RL	서버	터널	체크포인트
viva	0	0	-	all	8000+8002	pi05_viva_v5/250000
single_vla	-	0(e2e)	-	vla	8002	h100_endtoend/064000
eval_vlm_rl	0	-	S1~S4	vlm	8000	RL: dp_bc_small+resip64% 등

3. 실행환경 구축

클라이언트(데스크탑, Isaac Sim)와 서버(A100, VLM+VLA)로 나뉜다. 아래 순서대로 한번만 구축한다.

```
~/IsaacLab/scripts/lekiwi_nav_env/                                # 프로젝트 (클라이언트·서버 공통)
├─ vllm/
│   ├─ run_full_task.py                                           # viva / rl_hybrid 메인 루프
│   ├─ run_only_vla.py                                           # only-VLA
│   ├─ eval_vlm_rl.py                                            # VLM + RL expert
│   ├─ vlm_orchestrator.py                                       # VLM 지시 생성
│   ├─ vlm_prompts.py                                            # VLM 프롬프트
│   ├─ vla_inference_server.py                                    # VLA 서버 (포트 8002)
│   └─ train_v5.sh                                               # VLA(Pi0.5) 파인튜닝 래퍼
├─ launch_servers.sh                                             # VLM+VLA 서버 기동/종료
├─ setup.sh                                                      # 클라이언트 env (데이터 생성 매뉴얼
과 공유)
└─ secondserver/                                                # 서버 env 설치
    └─ setup_viva.sh                                             # vllm + lerobotpi0v2 + 모델 —
이 하나로 전부
    └─ env_vllm.lock.txt / env_lerobotpi0v2.lock.txt
~/miniconda3/envs/                                              # 서버: vllm, lerobotpi0v2 / 클라이언트:
env_isaaclab
[/home/jovyan/h100_deploy/base_model]                          # Pi0.5 base (setup_viva.sh 다운
로드, 프로젝트 밖)
```

3.1 conda env 설치

명령어

클라이언트 (데이터 생성 매뉴얼 setup.sh)

```
bash setup.sh
```

서버 (vllm + lerobotpi0v2 + 모델)

```
ssh A100  
bash secondserver/setup_viva.sh
```

secondserver/setup_viva.sh

[viva.sh — 노션 원문 참조]

secondserver/env_vllm.lock.txt

[env_vllm.lock.txt — 노션 원문 참조]

secondserver/env_lerobotpi0v2.lock.txt

[env_lerobotpi0v2.lock.txt — 노션 원문 참조]

3.2 모델 다운로드

명령어

VLM (Qwen3-VL-8B)

```
huggingface-cli download Qwen/Qwen3-VL-8B-Instruct
```

VLA base (Pi0.5, 파인튜닝 시작점)

```
huggingface-cli download lerobot/pi05_base --local-dir  
/home/jovyan/h100_deploy/base_model
```

3.3 VLA 파인튜닝

이 단계는 LeRobot 0.5.0 의 Pi0.5 를 우리 데이터로 파인튜닝하는 것이다. 먼저 짚을 점은 **lerobot 소스를 git clone 해서 .py 를 직접 고치는 일은 없다**는 것이다. lerobot 은 § 3.1 의 setup_viva.sh 가 pip install 로 v0.5.0 태그([pi] extra 포함)를 설치하고, 우리가 손대는 것은 우리 저장소의 코드와 학습 인자뿐이다.

lerobot 준비 (실제 설치는 § 3.1)

- lerobotpi0v2 env 에 lerobot 0.5.0(@v0.5.0 tag, [pi] extra) + fastapi · uvicorn 을 설치한다 (Python 3.12, torch 2.10, transformers 5.3).
- Pi0.5 가 google/paligemma-3b-pt-224(gated repo)에 의존하므로 huggingface-cli login 이 필수다.
- base 모델 lerobot/pi05_base(약 14GB)를 받아 학습 시 --policy.path 로 준다 (§ 3.2).

무엇이 '수정'인가 — lerobot 소스가 아니라 우리 코드와 인자다

1. **추론 서버 (vla_inference_server.py)**: config.json 의 type 으로 pi05 / pi0_fast 를 자동 감지하고, Pi0.5 는 make_pre_post_processors 파이프라인으로 정규화를 처리하며, 32D action 중 앞 9D 만 잘라 쓴다. (레거시 pi0_fast 경로에만 attention_mask 를 .bool()로 주고 validate_action_token_prefix=False 로 끄는 처리가 남아 있다. 현재 배포되는 Pi0.5 는 이 두 가지가 필요 없다.)
2. **학습 인자 (lerobot-train, train_v5.sh)**: rename_map 으로 데이터셋 카메라 키 front/wrist 를 Pi0.5 가 기대하는 base_0_rgb / left_wrist_0_rgb 로 매핑하고(3 번째 카메라 right_wrist 는 lerobot 이 -1 로 패딩),
max_state_dim · max_action_dim=32, chunk_size=50,
gradient_checkpointing=true(A100 40GB 초과 방지)로 준다.
3. **stats.json q01/q99 보정**: 수집 데이터가 navigate · carry 프레임에 쏠려 있어 그대로 정규화하면 approach 의 arm 값이 폭주한다(예: arm4 +0.534 → +5.60). 그래서 학습 전에 9 개 dim 의 q01/q99 를 실제 min/max ±5% margin 으로 넓혀준다. 이 보정을 안 하면 approach 동작을 학습하지 못한다.

전체 절차: ① 변환(convert_hdf5_to_lerobot_v3.py — HDF5 → LeRobot v3, fps 25, 9D state/action, front/wrist 비디오) → ② stats q01/q99 보정 → ③ 학습(train_v5.sh 의 lerobot-train — base=pi05_base, chunk 50, lr 5e-5, batch 8, 300K step, 배포 ckpt 는 250K) → ④ 서빙(§ 3.4 의 launch_servers.sh 가 vla_inference_server.py 로 :8002 /act 제공). 아래에 설계값과 실제 명령이 이어진다.

[파인튜닝 설계]

- VLA base 모델 및 경로 : lerobot/pi05_base (→ /home/jovyan/h100_deploy/base_model)

- 데이터셋 : lekiwi_viva_v5 978 ep (navigate 446 + carry 432 + approach 100, 209,036 frames)
chunk_size=50 / n_action_steps=50 : approach&lift 궤적이 길어 50 스텝이 필요하다 (VLA의 action 길이를 의미하는 step 으로 논문 기반 50~100 정도로 설정
학습시의 chunk size와 추론의 n_action_steps 는 같아야 함)

- 하이퍼파라미터 : batch_size=8 (A100 40GB + gradient_checkpointing 최대치) · bfloat16
· compile=false · lr=5e-5 · warmup=500 · decay=1e6 cosine · steps=300K

- 전처리 : max_state_dim=max_action_dim=32 (Pi0.5 패딩) · rename front->base_0_rgb, wrist->left_wrist_0_rgb
stats.json q01/q99 정규화 보정 필수 (모바일 매니퓰레이터 특성상 바퀴와 팔의 action 데이터가 전부 한 데이터셋의 action 에 들어가있는데 바퀴와 팔의 action 범위가 다르기 때문에 정규화가 잘못되면 바퀴에 맞춰 min,max 를 정하게 되면 arm 의 정규화가 폭주하여 바로 고장남)

명령어

변환

```
python convert_hdf5_to_lerobot_v3.py \
  --input demos/*.hdf5 \
  --output_root ~/lerobot_data/lekiwi_viva_v5 \
  --repo_id local/lekiwi_viva_v5 \
  --task_source full_task \
  --fps 25 --vcodec h264 --overwrite
```

파인튜닝

```
nohup lerobot-train \
  --dataset.repo_id=local/lekiwi_viva_v5 \
  --dataset.root=/home/jovyan/lerobot_data/lekiwi_viva_v5 \
  --dataset.video_backend=pyav \
  --policy.path=/home/jovyan/h100_deploy/base_model \
```

```

--policy.repo_id=local/pi05_lekiwi_v5 \
--policy.compile_model=false \
--policy.gradient_checkpointing=true \
--policy.dtype=bfloat16 \
--policy.chunk_size=50 \
--policy.n_action_steps=50 \
--policy.max_state_dim=32 \
--policy.max_action_dim=32 \
--policy.optimizer_lr=5e-5 \
--policy.scheduler_warmup_steps=500 \
--policy.scheduler_decay_steps=1000000 \
--batch_size=8 \
--steps=300000 \
--save_freq=10000 \
--log_freq=100 \
--eval_freq=0 \
--num_workers=16 \
--
rename_map='{ "observation.images.front": "observation.images.base_0_rgb",
"observation.images.wrist": "observation.images.left_wrist_0_rgb" }' \
--output_dir=outputs/train/pi05_viva_v5_$(date +%Y%m%d_%H%M%S) \
> ~/pi05_v5_train.log 2>&1 &

```

vllm/train_v5.sh

[v5.sh — 노션 원문 참조]

3.4 서버 서빙

[서버 자원 · 플래그]

GPU (A100 40GB) : VLM ~29.8GB(util 0.75) + VLA ~8GB = ~37.9GB. 0.5로 낮추면 KV cache OOM -> 0.75 필수

VLM serve : --dtype bfloat16 --max-model-len 4096 --gpu-memory-utilization 0.75 --trust-remote-code

VLA serve : config.json으로 pi05 / pi0_fast 자동 감지.
attention_mask.bool() + validate_action_token_prefix=False

launch_servers : all(VLM -> 60초 대기 -> VLA) | vlm | vla --checkpoint <path> | stop

[VLM 서빙 (vLLM · Qwen3-VL-8B-Instruct)]

서버 : vLLM OpenAI 호환 API 서버. setup_viva.sh가 만든 vllm env에서 실행

(launch_servers.sh vlm 이위 명령 래핑)

엔드포인트: http://localhost:8000/v1/chat/completions (chat) · /v1/models (health 확인)

클라이언트: vlm_orchestrator.py가 {model, messages:[base64 이미지 + 텍스트 프롬프트]} POST -> instruction 생성 (max_tokens 100~150, temperature 0.0)

플래그 : `--dtype bfloat16 / --max-model-len 4096` (프롬프트+이미지 토큰; 클수록 VRAM 낭비) / `--gpu-memory-utilization 0.75` (KV cache; 0.5=OOM, 0.75 초과=VLA VRAM 부족) / `--trust-remote-code` (Qwen3-VL 커스텀 코드)
자원 : ~29.8GB (bf16) - A100 40GB에서 VLA(~8GB)와 한 GPU 공유

[검증]

env 설치 : `setup_viva.sh` 끝에 `vllm / lerobotpi0v2` 버전 출력

서버 : `curl :8000/v1/models` (모델 JSON 보이면 OK) · `curl :8002/health` (응답 보이면 OK)

실행 결과 : 콘솔에 "Trial N → SUCCESS/TIMEOUT" + "Eval Summary ... S1/S2/S3 success rate" 출력

명령어

서버 기동

```
ssh A100
cd /home/jovyan/IsaacLab/scripts/lekiwi_nav_env
bash launch_servers.sh all --checkpoint
vllm/outputs/train/pi05_viva_v5_20260421_071948/checkpoints/250000/
pretrained_model
```

VLM 단독 서빙 (`vllm · launch_servers.sh vlm/all` 이 래핑하는 실제 명령)

```
conda activate vllm
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen3-VL-8B-Instruct --dtype bfloat16 \
  --port 8000 --max-model-len 4096 \
  --gpu-memory-utilization 0.75 --trust-remote-code
```

클라이언트 준비 (셀마다 1회 · GUI 실행, `--headless` 금지)

```
conda activate env_isaaclab
cd ~/IsaacLab/scripts/lekiwi_nav_env
export OMNI_KIT_ACCEPT_EULA=YES PYTHONUNBUFFERED=1
LEKIWI_USD_PATH=~/.Downloads/lekiwi_robot.usd
```

터널 (`viva / rl_hybrid`)

```
ssh -f -N -L 8000:localhost:8000 -L 8002:localhost:8002 A100
```

터널 (only-VLA)

```
ssh -f -N -L 8002:localhost:8002 A100
```


터널 (eval_vlm_rl)

```
ssh -f -N -L 8000:localhost:8000 A100
```

확인

```
curl -s localhost:8000/v1/models | head -c 80; echo  
curl -s localhost:8002/health
```

서버 비우기

```
ssh A100 "kill -9 -f '[v]llm'; kill -9 -f '[v]la_inference_server';  
fuser -k 8000/tcp 8002/tcp 2>/dev/null"
```

launch_servers.sh

[servers.sh — 노션 원문 참조]

vllm/vla_inference_server.py

[server.py — 노션 원문 참조]

4. VLM-VLA 시스템 코드 설명

VIVA 런타임은 4 개 파일이 맞물려 돈다. 핵심은 VLM(Qwen3-VL-8B)이 "무엇을 할지"를 자연어로 비동기 결정하고, VLA(Pi0.5)가 "어떻게 움직일지"를 동기로 매 스텝 생성하는 분리다. VLM 이 느려도(약 191ms) VLA 루프를 막지 않는다.

4.1 전체 흐름 (run_full_task.py)

시작할 때 한 번: ① VLM·VLA 서버 헬스 체크(:8000 /v1/models, :8002 /health)

→ ② classify 로 user_command 에서 source·dest 물체를 추출(1 회)

→ ③ env(Skill2Env 단일) 로드 + 카메라 3 개 생성(VLM 용 base RGB 1280×800, VLA 용 wrist 640×400, safety 용 depth 320×200)

→ ④ --mode 에 맞는 orchestrator 생성. --mode 는 viva(VLM+VLA 4-skill) / single_vla(VLA 만, VLM 은 자유 instruction 만) / rl_hybrid(S1 navigate·S3 carry 는 RL expert, S2·S4 는 VLA) 셋이다.

그 다음 매 스텝 아래 루프를 돈다.

한 스텝 안에서

(a) 카메라 캡처(depth 는 매 스텝, 비싼 RGB 는 VLM 호출이나 버퍼가 빌 때만)

→ (b) 전방 중앙 1/3 영역의 depth_min 계산

→ (c) 로봇 상태(contact·joint)를 orchestrator 에 넣고 코드 기반 전환을 판정

→ (d) 스킬별로 VLM 비동기 호출

→ (e) VLA 로 action 생성

→ (f) depth safety layer

→ (g) 스킬 전환 시 VLA 버퍼 리셋 + BRAKE → (h) env.step 순서다.

4.2 4-skill 상태머신과 전환

스킬은 NAVIGATE → APPROACH_AND_LIFT → CARRY → APPROACH_AND_PLACE → DONE 순으로 진행한다. S2 중 장애물을 만나면 NAVIGATE 로 돌아갔다가 다시 진입하고 (복귀용 _interrupted_skill 로 원래 스킬을 기억), S4 도 같은 식으로 CARRY 를 거쳐 복귀한다.

전환 트리거는 두 종류다.

- **VLM 기반 (S1→S2, S3→S4)**: VLM 이 마지막 줄에 정확히 TARGET_FOUND 를 출력하면 전환한다. 단 전방 중앙 depth 가 1.2m 보다 멀면 "아직 멀다"고 거절하고 현재 스킬을 유지하며, 다음 프롬프트에 거절 사실을 한 번 알려 재오인식을 억제한다.

- **코드 기반 (S2→S3, S4→DONE)**: joint 위치로 판정한다. S2→S3 는 contact 가 있고 팔 관절이 LIFTED_POSE 범위에 들어오면, S4→DONE 은 그리퍼가 열리고 비접촉이 10 스텝 연속이면 전환한다.

타임아웃·재시도로 S2/S4 는 각 10000 스텝 timeout + 최대 3 회 재진입(왕복 무한루프 방지), 전체 max_total_steps 는 6000 이다.

4.3 vlm_orchestrator.py — VLM 두뇌

orchestrator 는 VLM 을 호출해 "지금 무엇을 할지"를 정하고 4-skill 상태머신을 굴리는 VIVA 의 두뇌다. viva·rl_hybrid 는 VIVAOrchestrator 를, single_vla 는 RelativePlacementOrchestrator 를 쓴다.

classify_user_request() — 시작 시 text-only VLM 을 1 회 호출해 user_command 를 {mode, source, dest}로 파싱한다. 물체명은 _normalize_obj(소문자 변환·언더스코어/하이픈→공백·공백 정규화)로 VLA 학습 라벨과 정확히 맞춘다("Medicine Bottle" → "medicine bottle"). 파싱에 실패하면 기본값(relative_placement / medicine bottle / red cup)으로 fallback 한다.

비동기 instruction 생성 — VLM 이 느려도(약 191ms) VLA 루프를 막지 않게 호출을 별도 스레드에서 돌린다.

- query_async(rgb)가 daemon thread 를 띄워 query_instruction()을 호출하고 즉시 리턴한다. 결과는 lock 아래 _latest_instruction 에 저장되고, 메인 루프는 instruction 프로퍼티로 매 스텝 최신값만 읽는다.

- _pending 플래그로 동시에 한 호출만 허용해 자동 throttle 하고, vlm_interval(기본 30, viva 실행은 100)로 호출 빈도를 제한한다.

- _generation 카운터: trial 이 리셋되면 ++ 해서, 리셋 전에 보낸 요청이 늦게 도착하면 (stale) 그 응답을 버린다.

- query_instruction 은 POST /v1/chat/completions(timeout 8 초, max_tokens 150, temperature 0)다.

프롬프트 빌드 (_build_vlm_payload) — 현재 스킬에 맞는 system prompt(S1 navigate / S3 carry / S2·S4 obstacle monitor)와 카메라 이미지로 payload 를 만든다. S2/S4 에서 장애물로 복귀하는 중이면 우회 힌트를, TARGET_FOUND 가 거부됐으면 거부 사실과 거리를 한 번 끼워 넣어(target_reject_hint) 같은 오인식을 억제한다.

응답 파싱·전환 (_process_vlm_response)

- 마지막 비어있지 않은 줄이 정확히 TARGET_FOUND 일 때만 전환을 시도한다
(reasoning 줄의 "not TARGET_FOUND" 같은 문구에 오발동하지 않게). 단 전방 중앙 depth 가 1.2m 보다 멀면 "아직 멀다"고 거부한다.
- S2/S4: OBSTACLE → 장애물 처리, CONTINUE → obstacle_cleared=True, 아무것도 안 나오면 안전하게 default CONTINUE.
- S1/S3: NAVIGATE_COMMANDS / CARRY_COMMANDS 집합과 먼저 exact match, 안 되면 substring 최장일치로 매칭하고, 그래도 실패하면 이전 instruction 을 유지한다.

장애물 복귀 (_handle_obstacle) — S2 중 장애물을 만나면, 이미 물체를 들었으면 (contact + lifted) CARRY 로, 빈손이면 NAVIGATE 로(빈손 회피) 보낸다. S4 는 항상 CARRY 로 내려간다. _interrupted_skill 에 원래 스킬을 기억해 회피가 끝나면 그 스킬로 복귀한다. 또 depth 상태로, CONTINUE 이후 멀어졌다가(>0.5m) 다시 가까워지면(<0.3m) 새 장애물로 보고 obstacle_cleared 를 리셋한다.

safety_layer_active — NAVIGATE/CARRY 에서만 True 다. 그래서 run_full_task 의 depth safety(전진 차단)가 S1/S3 에서만 작동한다. S2/S4 는 목표물에 접근해야 하므로 depth 경고를 차단이 아니라 VLM 텍스트로만 전달한다.

RelativePlacementOrchestrator (single_vla 용) — 상태머신 없이 INSTRUCT 프롬프트로 약 0.3Hz 비동기 VLM 호출을 돌려 자유 자연어 instruction 하나를 만든다. VLM 이 "done"을 내면 종료한다. VLA 만 쓰는 비교군 모드용이다.

4.4 vla_inference_server.py — VLA 서버 (:8002)

- **엔드포인트:** /health, /act(POST). 요청은 base·wrist 이미지(JPEG base64) + state 9D + instruction 이고, 응답은 action chunk(horizon×9)다.
- **모델 로드:** config.json 의 type 으로 pi05/pi0_fast 를 자동 감지한다. Pi0.5 는 PI05Policy.from_pretrained + make_pre_post_processors 로 전처리(rename front→base_0_rgb·wrist→left_wrist_0_rgb → 정규화 → 토큰화)와 후처리(역정규화) 파이프라인을 만든다.

- **추론**: 이미지+state+instruction → predict_action_chunk → (T, 32) → 클라이언트가 앞 9D 만 잘라 쓴다.

- **클라이언트 버퍼(n_use)**: VLAClient 는 chunk 를 한 번 받아 n_use 개만 실행한 뒤 다시 요청한다 — NAVIGATE/CARRY 는 10(receding, 반응성 ↑), APPROACH/PLACE 는 50(full chunk, grasp·place 정밀 협응). 스킬 전환 때 버퍼를 리셋한다.

4.5 vlm_prompts.py — 프롬프트

VLM 이 출력하는 문자열이 곧 VLA 의 instruction 이 되므로, 프롬프트는 VLM 이 VLA 학습 task 라벨과 정확히 일치하는 문자열만 내도록 설계한다. 파일에는 7 개 프롬프트가 있다.

CLASSIFY (시작 1 회, text-only) — task parser. user_command 에서 source_object·dest_object·mode 를 뽑아 JSON 만 출력한다. mode 는 relative_placement(A 를 B 옆에) 또는 single_pickup(A 만 집기)이다.

INSTRUCT (single_vla 비교군) — commander. 이미지를 보고 ONE short English instruction 을 낸다. 상황별 가이드(물체가 안 보이면 탐색, 멀면 접근, 가까우면 grasp, 들고 있는데 dest 가 안 보이면 탐색, dest 가 보이면 place, 완료면 "done")를 따르며, 영어 한 문장·방향 구체·reasoning 금지다.

NAVIGATE_LEGACY (ResiP Navigate 레거시) — FORWARD/BACKWARD/TURN_LEFT/TURN_RIGHT/STOP 단어와 COMMAND_TO_DIRECTION 매핑. 현재 viva 에서는 쓰지 않는다.

VIVA S1 Navigate / S3 Carry (방향 commander) — 구조가 거의 같다.

- 입력: 대상 물체(target 또는 source·dest), robot_status(관절·그리퍼·depth 경고를 담은 텍스트), prev_command, 카메라 이미지.

- 출력 형식: 먼저 장면을 1~2 문장으로 묘사하고, 마지막 줄에 정확한 명령 문자열 하나를 낸다.

- 명령: S1 은 "navigate forward / backward / strafe left / strafe right / turn left / turn right **with arm tucked**" 6 개 + "TARGET_FOUND". S3 은 "carry forward /

backward / left / right / turn left / turn right **holding the object**" 6 개 +

"TARGET_FOUND".

- Safety layer 안내: 0.3m 앞에 장애물이 있으면 forward 는 자동 차단(BLOCKED)되지만 backward·strafe·turn 은 항상 동작한다고 프롬프트가 직접 알려, 막혔을 때 탈출 명령을 내게 한다.

- self-occlusion 주의: depth 경고가 로봇 자기 팔·그리퍼(S3 은 든 물체까지)일 수 있으니 시각으로 확인하고, 자기 팔이면 forward 를 유지하라고 안내한다.

- TARGET_FOUND 는 보수적으로: 물체가 가깝고(디테일이 보이고·프레임 하단에 있고·사이 바닥이 적고) 중앙에 있을 때만 낸다. 작거나·멀거나·먼 벽 근처거나·한쪽으로 치우치면 더 다가가거나 돌려 중앙에 오게 한 뒤에 낸다.

- 명령은 verbatim(정확한 문자열 그대로, 단어 추가·삭제·재배열 금지)으로 내야 한다 — 이것이 VLA 학습 라벨과 exact match 되는 핵심이다.

VIVA S2 Approach & Lift / S4 Approach & Place (obstacle monitor) — depth 경고가 떴을 때만 호출된다.

- 역할: 가까운 물체가 목표(source/dest)인지 예상 못한 장애물(벽·가구·엉뚱한 물체)인지 판별한다.

- self-occlusion 강조: 로봇 자기 5-DoF 팔·그리퍼(S4 는 든 물체까지)가 카메라 전경을 자주 채우는데, 이건 장애물이 아니므로 항상 CONTINUE 다.

- Decision rules(순서대로, 첫 매치에서 멈춤): ① 전경이 자기 팔·그리퍼면 CONTINUE → ② 목표 물체가 일부 가려져도 보이면 CONTINUE → ③ 벽·큰 가구·엉뚱한 물체가 중앙을 채우고 팔이 아니면 OBSTACLE → ④ 애매하면 CONTINUE(잘못 후퇴하는 비용이 더 크다).

- 출력: 정확히 CONTINUE 또는 OBSTACLE 한 단어만(설명·문장부호·따옴표 없이).

instruction 고정 — S1·S3 는 위 방향 명령이 그대로 VLA instruction 이 되지만, S2·S4 는 VLM 이 방향이 아니라 장애물 여부만 보므로 instruction 을 코드가 고정한다. 스킬 진입 시 _transition_to 가 S2 는 "approach and lift the {canonical}"로 (canonicalize_approach_target())가 "the blue medicine bottle" 같은 수식어를 떼어

학습 라벨 "approach and lift the medicine bottle"에 맞춤), S4 는 "place the {source} next to the {dest}"로 설정한다.

4.6 비동기 타이밍 — 느린 두뇌, 빠른 손

VLM 은 2 단계로 호출된다. `classify(text→물체, 약 250ms, 시작 1 회)`와 `instruct(image→instruction, 약 210ms, 루프 중 반복)`다. 실측은 VLM 약 191ms(비동기), VLA 27~58ms(동기), 전체 약 6.4Hz 다. VLM 은 수십 스텝에 한 번 방향만 바꾸고 VLA 가 매 스텝 그 방향으로 정밀 모션을 만든다. 비동기 호출이 동기 루프를 막지 않기 때문에 느린 VLM 과 빠른 VLA 가 한 루프에서 공존한다. GPU 는 VLM 약 19.8GB + VLA 약 7.7GB 로 약 27.5GB/40GB 를 쓴다.

메인 루프가 VLM 을 언제 부르나 — 매 스텝 `should_query` 를 계산해 정한다. `total_steps` 가 `vlm_interval` 의 배수인 주기마다, 또는 `depth` 가 처음 `safety_dist` 안으로 들어온 순간 (edge)에 즉시 1 회 부른다. `depth_close edge` 는 `interval` 을 기다리지 않고 바로 트리거해 장애물 반응을 빠르게 한다. `query_async` 를 부른 뒤 메인 루프는 곧바로 `instruction` 을 읽어 VLA 를 돌리고 VLM 응답을 기다리지 않는다.

백그라운드 worker 가 하는 일 — `query_async` 는 `_pending` 이면 즉시 빠져 동시에 한 호출만 돈다. 띄운 worker 는 현재 `_generation` 을 캡처하고, VLM HTTP 를 호출하고, 응답이 와도 `generation` 이 바뀌었으면(그 사이 trial 리셋) 버리고, 아니면 lock 아래 `_latest_instruction` 을 갱신하되 "done"이면 덮지 않고, finally 로 `_pending` 을 푼다. `generation` 은 worker 시작·응답 도착·`query_instruction` 내부 3 곳에서 확인해 stale 응답이 새 trial 을 오염시키지 않게 한다.

한 스텝의 타임라인 — 느린 VLM 이 빠른 VLA 를 막지 않는 모습을 시간순으로 보면:

```
step N    : query_async 띄움 → VLM 백그라운드 시작. instruction=직전값, VLA 실행
step N+1  : _pending=True 라 query_async skip. instruction=직전값, VLA 실행
...       : VLM 도는 1~2 스텝 동안 로봇은 직전 지시로 계속 움직임
VLM 완료 : worker 가 _latest_instruction 갱신
step N+2  : instruction=새 값, VLA 가 새 지시 반영
```

VLM 이 느려도 VLA 는 한 스텝도 멈추지 않고, 지시어만 1~2 스텝 늦게 갱신된다.

S2/S4 는 장애물만 비동기로 묻는다 — APPROACH_AND_LIFT/PLACE 에서는 방향이 아니라 장애물 여부만 보므로 query_async 대신 query_obstacle_check_async 를 쓴다. depth warning(depth_min 이 safety_dist 미만)이 떴고 obstacle_cleared 가 아닐 때만 부른다. 결과는 _latest_instruction 을 덮지 않고 obstacle_cleared · 장애물 복귀 플래그로만 반영한다. instruction 은 스킬 진입 시 코드가 "approach and lift the \{canonical\}" 식으로 고정해 두기 때문이다.

5. VIVA 실행

명령어

서버 + 터널

```
ssh A100 'cd /home/jovyan/IsaacLab/scripts/lekiwi_nav_env && bash
launch_servers.sh stop && bash launch_servers.sh all --checkpoint
vllm/outputs/train/pi05_viva_v5_20260421_071948/checkpoints/250000/
pretrained_model'
ssh -f -N -L 8000:localhost:8000 -L 8002:localhost:8002 A100
```

실행

```
python vllm/run_full_task.py --mode viva --difficulty easy \
--user_command "find the blue medicine bottle and place it next to the
red cup" \
--object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
--dest_object_usd
~/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_
oz_cup/model_clean.usd \
--scene_idx 1302 --scene_scale 0.6 --num_trials 10 --vlm_interval 100
```

vllm/run_full_task.py

[task.py — 노션 원문 참조]

vllm/vlm_orchestrator.py

[orchestrator.py — 노션 원문 참조]

vllm/vlm_prompts.py

[prompts.py — 노션 원문 참조]

6. only VLA

[전체 task를 endtoend로 수집한 데이터셋으로 학습시킨 VLA]

목적 : viva(VLM 지시문 + 스킬분할 VLA) vs 단일 정책 end-to-end 직접 비교

데이터 : full 텔레옴 easy/middle(같은 방)+hard(다른 방) · ~4000 frame/ep ·

full-task instruction 1개

"find the medicine bottle and place it next to the red cup"

데이터 경로: ~/lerobot_data/lekiwi_full_teleop_combined (repo_id

local/lekiwi_full_teleop_combined)

aug : per-dim adaptive noise ($\sigma=0.5 \times \text{delta_std}$, arm 0.0005~0.003 /
base 0.02~0.05)

· state noise=0 → 96 ep / 383,584 frame

학습 : batch16 · lr7e-5(8배치 보정) · chunk_size50 · decay60K ·

warmup1000 · 80K steps

체크포인트: outputs/h100_endtoend/checkpoints/064000

추론 : run_only_vla, "blue" 없는 원문 instruction

명령어

데이터 수집 (full 텔레옴 · 데스크탑 GUI · easy/middle=같은 방, hard=다른 방, ~4000 frame/ep)

```
PYTHONUNBUFFERED=1 python vllm/record_teleop_scene.py --skill full --  
difficulty easy \
```

```
--instruction "find the medicine bottle and place it next to the red  
cup" \
```

```
--object_usd
```

```
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
```

```
--dest_object_usd
```

```
~/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_  
oz_cup/model_clean.usd \
```

```
--scene_idx 1302 --scene_scale 0.6 --num_demos 12
```

```
PYTHONUNBUFFERED=1 python vllm/record_teleop_scene.py --skill full --  
difficulty middle \
```

```
--instruction "find the medicine bottle and place it next to the red  
cup" \
```

```
--object_usd
```

```
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
```

```
--dest_object_usd
```

```
~/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_  
oz_cup/model_clean.usd \
```

```
--scene_idx 1302 --scene_scale 0.6 --num_demos 12
```

```

PYTHONUNBUFFERED=1 python vllm/record_teleop_scene.py --skill full --
difficulty hard \
  --instruction "find the medicine bottle and place it next to the red
cup" \
  --object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
  --dest_object_usd
~/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_
oz_cup/model_clean.usd \
  --scene_idx 1302 --scene_scale 0.6 --num_demos 12

```

변환 (lerobot050 env · full_task = 전체 task 1개 instruction, 서브태스크 분할 X)

```

conda activate lerobot050
python convert_hdf5_to_lerobot_v3.py \
  --input "demos/scene_teleop_full_*.hdf5" \
  --output_root ~/lerobot_data/lekiwi_full_teleop_combined \
  --repo_id local/lekiwi_full_teleop_combined \
  --task_source full_task \
  --full_task_text "find the medicine bottle and place it next to the
red cup" \
  --fps 25 --vcodec h264 --overwrite

```

파인튜닝 (서버 lerobotpi0v2 env · e2e Pi0.5 · batch16/lr7e-5/80K → outputs/h100_endtoend)

```

lerobot-train \
  --dataset.repo_id=local/lekiwi_full_teleop_combined \
  --dataset.root=~/lerobot_data/lekiwi_full_teleop_combined \
  --policy.path=/home/jovyan/h100_deploy/base_model \
  --policy.repo_id=local/pi05_h100_endtoend \
  --policy.compile_model=false \
  --policy.gradient_checkpointing=true \
  --policy.dtype=bfloat16 \
  --policy.chunk_size=50 \
  --policy.n_action_steps=50 \
  --policy.max_state_dim=32 \
  --policy.max_action_dim=32 \
  --policy.scheduler_decay_steps=60000 \
  --policy.optimizer_lr=7e-5 \
  --batch_size=16 \
  --steps=80000 \
  --save_freq=8000 \
  --log_freq=100 \
  --eval_freq=0 \
  --num_workers=8 \
  --tolerance_s=0.1 \
  --
rename_map='{ "observation.images.front": "observation.images.base_0_rgb",
"observation.images.wrist": "observation.images.left_wrist_0_rgb" }' \
  --output_dir=outputs/h100_endtoend

```

서버 + 터널 (vla · e2e ckpt)

```
ssh A100 'cd /home/jovyan/IsaacLab/scripts/lekiwi_nav_env && bash
launch_servers.sh stop && bash launch_servers.sh vla --checkpoint
/home/jovyan/h100_endtoend_backup/outputs/h100_endtoend/checkpoints/
064000/pretrained_model'
ssh -f -N -L 8002:localhost:8002 A100
```

추론 (instruction 은 VLA 학습 문자열과 정확 일치)

```
python vllm/run_only_vla.py --difficulty easy \
  --instruction "find the medicine bottle and place it next to the red
cup" \
  --object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
  --dest_object_usd
~/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_
oz_cup/model_clean.usd \
  --scene_idx 1302 --scene_scale 0.6 --num_trials 10
```

vllm/run_only_vla.py

[vla.py — 노션 원문 참조]

7. VLM + RL

eval_vlm_rl 은 VLM 이 스킬 전환, 방향 지시만 말고 각 지시문에 대한 저수준 제어는 이전에
생성해둔

Skill 별 expert 가 진행하는 방식이다. 이는 viva 와 직접 비교하는 baseline 이다.

S1 navigate : DP BC + ResiP (26D obs). VLM 방향 지시 → direction_cmd, arm 은
force_tucked

S2 approach&lift : DP BC + ResiP (30D obs)

S3 carry : DP BC + ResiP (39D obs)

S4 approach&place : DP BC + ResiP (55D obs)

스킬 전환 : S1→S2 / S3→S4 = VLM TARGET_FOUND (중앙 depth>1.2m 면 거부) .

S2→S3 / S4→DONE = 코드 pose 판정

서버 : VLM(8000)만 필요, VLA 불필요

명령어

서버 + 터널

```
ssh A100 'cd /home/jovyan/IsaacLab/scripts/lekiwi_nav_env && bash
launch_servers.sh stop && bash launch_servers.sh vlm'
ssh -f -N -L 8000:localhost:8000 A100
```

실행

```
python vllm/eval_vlm_rl.py --difficulty easy \
  --nav_dp_checkpoint
checkpoints/dp_bc_nav_skill2_v4/dp_bc_epoch300.pt \
  --nav_resip_checkpoint
checkpoints/resip_nav_tucked_v4/resip_nav_best.pt \
  --dp_checkpoint checkpoints/dp_bc_small/dp_bc_epoch150.pt \
  --resip_checkpoint "backup/appoachandlift/resip64%.pt" \
  --carry_dp_checkpoint checkpoints/dp_bc_carry_v4/dp_bc_epoch300.pt \
  --carry_resip_checkpoint
checkpoints/resip_carry_v6/resip_carry_iter240.pt \
  --place_dp_checkpoint
checkpoints/dp_bc_skill3_55d_fixed_1e-4/dp_bc_epoch500.pt \
  --place_resip_checkpoint checkpoints/resip_s3_v25/resip_iter80.pt \
  --object_usd
~/isaac-objects/mujoco_scanned_objects/models/5_HTP/model_clean.usd \
  --dest_object_usd
~/isaac-objects/mujoco_scanned_objects/models/ACE_Coffee_Mug_Kristen_16_
oz_cup/model_clean.usd \
  --scene_idx 1302 --scene_scale 0.6
```

vllm/eval_vlm_rl.py

[rl.py — 노션 원문 참조]